PHYS30762 Object Oriented Programming in C++

Assignment 5 Report

Kuba Kane
11019699

Department of Physics and Astronomy
The University of Manchester

April 2026

Contents

1 Program summary **1**
1.1 Electron class . . . 1
1.2 Muon class . . . 1
1.3 Tau class . . . 3
1.4 Neutrino class . . . 3
1.5 Main cpp file . . . 3

2 Code design **4**
2.1 Four momentum class . . . 4
2.2 Lepton class . . . 5
2.3 Rule of 0/3/5 . . . 6

3 References **7**

4 Delegation to Generative AI **8**
4.1 Disclosure of Delegation to Generative AI . . . 8
4.2 GAI prompts . . . 8

Note to markers: For version history please see the README file for different branches on the Github repository for assignment 5, as this contains planning and content for this report.

1. Program summary

This program is based around a `Lepton` class, which is an abstract base class from which `Electron`, `Muon`, `Tau`, and `Neutrino` are derived classes. The `Lepton` class contains a unique pointer to a `FourMomentum` object, whose class has energy, x momentum, y momentum, z momentum, and rest mass, as data members (all measured in MeV). The `Lepton` class also contains a rest mass and relative charge, as well as a boolean expression which if true represents an antilepton by flipping the charge data member. The `Lepton` class contains 2 virtual member functions which makes it abstract (along with its virtual destructor). The first is the `info()` function which is used to print out additional properties of derived classes, and `has_decayed()` which is set true if one of the derived leptons have decayed. `Lepton` contains `base_info()` to print out the properties it carries (four momentum, rest mass, and relative charge).

The derived classes inheret all these properties, but also have their own data members and member functions. These additional properties are printed by overriding the `info()` function (using `base_info()` to print the properties from `Lepton`), and are used to determine the outcome of `has_decayed()`. Figure 1 is a UML diagram which shows the heirarcy of the derived classes, and their properties. All of the derived classes use `static constexpr` to store their mass and charge as constants which cannot be modified during compilation or run time, which are printed via `base_info()`.

The lepton class also contains 2 friend functions - one to sum the four momenta of 2 leptons, and the other to compute their dot product.

1.1. Electron class

This class introduces an interaction with a detector, primarily through calculating the shower depth in a tungsten (W) detector via the member function `shower_depth_in_tungsten()`. This function calculates the shower depth X using the equation

$$X = X_0 \frac{ln(E_0/E_c)}{ln2}, \tag{1}$$

where X_0 is the radiation length, E_c is the critical energy, and E_0 is the initial energy of the electron (from its `FourMomentum`). The first two of these are set within the member function, matching the values for tungsten. E_0 is accessed via the pointer to the `FourMomentum` from the Lepton base class. `Electron` also has the function `shower_contained(const double detector_length_in)` which, given a detector length, determines if the shower is contained within the detector.

1.2. Muon class

The `Muon` class introduces the data member `time_in_detector()` which, if greater than the constant `MUON_MEAN_LIFETIME` stored as a private data member in the class, sets `has_decayed()` to be true. A `validate_time()` static function is used to check that the time entered by the user is not negative or greater than 1 second.

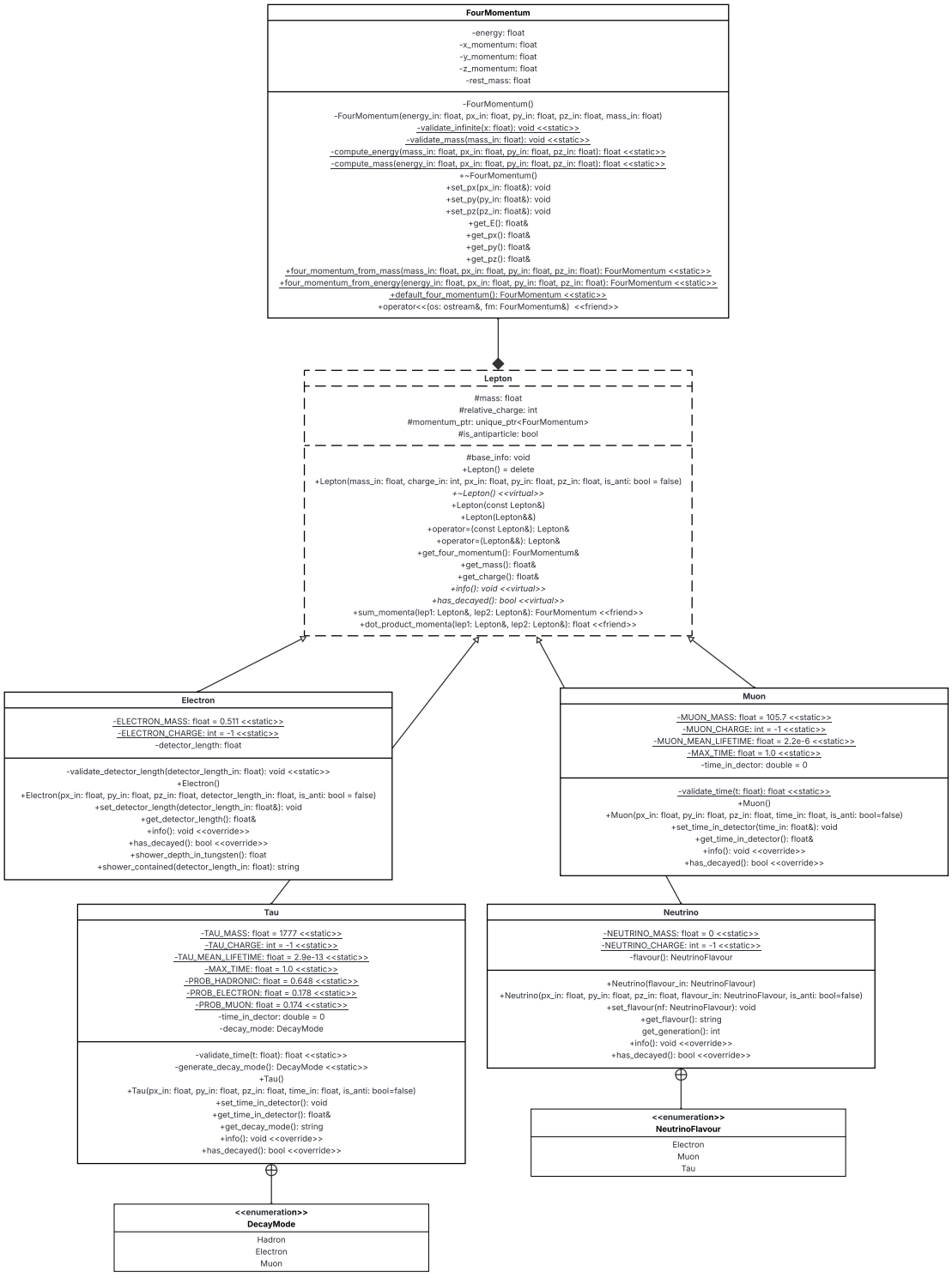


Figure 1: UML diagram of assignment 5 program, depicting all the classes in the program. Access online via `https://lucid.app/lucidchart/1bbac6b7-964d-4fed-86ef-e02a48e03f28/view`. Enumerator classes represented as nested classes, and FourMomentum as composition of Lepton (the instance in FourMomentum lives and dies via the unique pointer in the Lepton class). Electron, Muon, Tau, Neutrino are derived classes of Lepton. Virtual functions are denotes in italics, while static functions are underlined. Friends functions are denoted using «friend».

1.3. Tau class

The `Tau` class uses `time_in_detector()` and with `TAU_MEAN_LIFETIME` to handle the `has_decayed()` function, akin to the `Muon` class. Unlike the `Muon` class, however, it takes into account the different decay modes of taus. Using the fact that 64% of the time a taus' decay products are hadronic, 17.8% of the time they involve electrons, and 17.4% of the time they involve muons, a private member function `generate_decay_mode()` uses a basic Monte Carlo simulation to set the `decay_mode` data member to one of the three named values in the enumerator class defined for the decay modes. The getter `get_decay_mode` uses a switch to return the decay products as a string, differing for each named value in the enumerator class, and this is used in the overriden `info()` function.

1.4. Neutrino class

In the neutrino class, the idea of lifetime is discarded and `has_decayed` is always set to false due to the stability of neutrinos. An enumerator class `NeutrinoFlavour` is used with three named values corresponding to the three neutrino flavours. The neutrino flavour is set as a private data member which can be specified via the neutrino constructor or flavour setter. Since a neutrino cannot be flavourless, the default constructor also requires flavour as an input. The `info()` function prints the flavour using a switch returning a string dependent on the named value in the enumerator class, as well as the flavour generation, again dependent on the named value.

1.5. Main cpp file

The main file is split into 6 parts. The first part creates a vector of different leptons called `particles` via unique pointers to the base `Lepton` class, and then prints out all of their info using an iterator through all elements in the vector. The second part accesses 2 electrons by dereferencing the first two elements and sums their four momenta. The third part accesses the anti electron and anti muon and takes the dot product of their four momenta. The fourth part creates a unique pointer to an electron and moves it to a second pointer. The fifth part creates a shared pointer for a tau lepton and assigns into to 2 other pointers. The final part creates 2 vectors of shared particles ("detectors"), and gives them the same tau lepton pointer, which is shown by printing out all elements in both of the vectors.

2. Code design

2.1. Four momentum class

The `FourMomentum` class was changed so that, unlike in assignment 4 where its only data member was a unique pointer to a 4-vector containing energy E and the 3-momentum components (px, py, pz), it contained each component as a separate data member. This made the object easier to deal with when performing checks for invariant mass, since it relates to the four momentum components via the equation

$$m^2 = E^2 - \mathbf{p}^2, \tag{2}$$

where m is the particle's rest mass (also known as invariant mass) and $\mathbf{p}$ is the 3-momentum.

The desire to include invariant mass originates from distinguishing different derived classes of `Lepton` with the particle mass, and therefore any `FourMomentum` object which the derived classes deal with must give an invariant mass which matches the rest mass of that specific particle. The issue with `FourMomentum` class as in assignment 4 was that it required the user to specify E, px, py, pz in one go to generate the 4-vector. This was resolved by making the FourMomentum class have E, px, py, pz as data members themselves, allowing the 3-momentum to be set by the user, and then using the invariant mass equation to determine the energy and define the FourMomentum object.

Setters for E, px, py, pz would allow the user to modify the `FourMomentum` object and change the invariant mass, completely breaking checks for invariant mass. This was first resolved by removing all setters from the `FourMomentum` class, and only allowing (px, py, pz) to be set via the `Lepton` base class (through its constructors). Since `Lepton` contained a mass data member, the energy could then be computed using equation 2, creating a valid `FourMomentum` object. Individual setters for px, py, pz were then possible by re-computing the energy.

This however made the `FourMomentum` object completely redundant, as it was mostly controlled by the `Lepton` class, and the user could make a `FourMomentum` object which completely broke these checks. To avoid this, the code was redesigned to keep the control of the `FourMomentum` object completely within the `FourMomentum` class, thus disallowing a `FourMomentum` object to be created which didn't obey equation 2. One idea was to make `Lepton` a friend class of `FourMomentum` and making the constructors private such that a `FourMomentum` object could not be created outside of that class. This didn't work, however, due to the unique pointer in the `Lepton` class, as it makes a new FourMomentum object not associated with the Lepton class, which wouldn't be possible.

The solution used was to redesign the `FourMomentum` class to also include a mass as a data member which could be used to perform check. The `Lepton` class can then pass its mass into the constructor of the form (m, px, py, pz). This is more robust, as if the user wanted to make another base class, e.g. quarks, using `FourMomentum`, they wouldn't need to worry about checking that invariant mass is conserved, as all checks would be handled by the `FourMomentum` class. One issue that came up was with the friend function which summed the four momenta of 2 leptons, as this required a constructor in `FourMomentum` of the form (E, px, py, pz), which could not be overloaded as it contained the same elements of the same type as the (m, px, py, pz) constructor.

To get around this, a constructor of the form (E, px, py, pz, m) was made private, and two static functions `four_momentum_from_mass` and `four_momentum_from_energy` were

introduced, where the latter allowed for the four momenta sum to be computed. This also gave the benefit that a user, who may not know the mass of a particle but does know its initial energy, could find its invariant mass and still construct a `FourMomentum` object (this is technically what is done when 2 four momenta are summed - a new combined invariant mass is calculated so that the new summed `FourMomentum` object still obeys equation 2). Setters for px, py, pz were added in the `FourMomentum` class, which assumed the invariant mass was conserved (and energy changed) to keep in line with requirements for the `Lepton` class. Setters for mass and energy were not possible as both would allow the user to modify the rest mass.

The overload `<<` operator ensures that only E, px, py, pz are printed out when a `FourMomentum` object is called - the rest mass can be thought of as a "hidden" property.

2.2. Lepton class

In the main function for this assignment, we are never required to deal with just a lepton, but rather some specific type of lepton. Also, there is no need to work with lepton object - anything that is a lepton has a more specific particle category (e.g. electron, muon) with defined properties. Making `Lepton` an abstract class therefore makes most sense. To make `Lepton` an abstract class, at least one virtual function needs to be pure virtual. To adhere to this required making the `info()` function print only additional features which `Lepton` doesn't have, this allowing the info function to be pure virtual. To avoid repeating code, the `base_info()` function was added to print out features which `Lepton` does have, which can then be reused in all derived classes.

`constexpr` was used for the mass and charge of all derived classes of `Lepton`. It specifies that the mass and charge values of the derived object can be evaluated at compile-time, i.e. it doesn't change during run-time. It makes it obvious that mass and charge are physical constants and there are no changes of setting these incorrectly elsewhere in the program. The mass and charge were set to private data members, to keep it separate from the getters `get_mass` and `get_charge`, giving the user only one way to access them, consistent across all derived classes.

The other property of `Lepton` is the four momentum. The lepton class uses a **unique** smart pointer called `momentum_ptr`, which points to a four momentum object. It is a unique pointer rather than shared because `FourMomentum` is a property which should be unique to a lepton. Shared pointers allow multiple pointers to point to the same object, so if the user wanted to change the `FourMomentum` object of one lepton, it would also change it for all leptons which point to the same `FourMomentum`. Also the FourMomentum of one object cannot be deleted until all shared pointers become out of scope, making it more difficult to deal with four momenta sums or dot products.

This assignment required moving the four momenta sum and dot product to be friend functions of `Lepton` instead of `FourMomentum`. It makes sense that these operators are not non-member functions, as this would require unecessary calling of the `Lepton` class getters / setters. So that these functions can instead access the private data members of `Lepton` class, i.e. the four momentum, these should be friend functions of Particle class. This makes more sense than having them as member functions of `Letpon`, since being a member function would require one `FourMomentum` to call another FourMomentum object with which it wishes to perform a sum or dotProduct operation. This would imply that the two FourMomentum objects are not equal in some ways.

To deal with antileptons without creating separate classes for them, a boolean parameter

in the `Lepton` base class `is_antiparticle` was introduced, which is true if it is an antiparticle, and false if it is not. The charge is then set to be negative of the lepton's default charge (set via `constexpre`) if its an antilepton. The user can never set the charge - the derived classes specify this, and setters for charge do not exist. If the user does not specify `is_antiparticle` in the constructor, it defaults to false, i.e. assumes it's a lepton and not an antilepton. Default constructors for all derived classes assume the given particle is not anti.

2.3. Rule of 0/3/5

The `FourMomentum` object has the data members E, px, py, pz which are all doubles. As there are no pointers involved (or any other objects of non-class type), we use the rule of 0. As such, the `FourMomentum` object does not need to have user-defined special member functions. Since the `Lepton` class involves a smart pointer, we must implement the rule of 3 at least. As the assignment requires `std::move` of `Electron` (derived from `Lepton`), we need move semantics and so must implement the rule of 5. All derived classes inherit the constructors and assignment operators defined in `Lepton` base class, and since they do not have any pointers themselves, we use rule of 0 for derived classes.

3. References

The following sources were used in the research for this assignment.

[1] https://www.geeksforgeeks.org/cpp/pure-virtual-functions-and-abstract-classes/

[2] https://en.cppreference.com/w/cpp/language/rule_of_three.html

[3] https://www.geeksforgeeks.org/cpp/understanding-constexper-specifier-in-cpp/

[4] https://www.geeksforgeeks.org/cpp/sqrt-sqrtl-sqrtf-cpp/

[5] https://www.geeksforgeeks.org/cpp/enum-classes-in-c-and-their-advantage-over-enum-datatype/

[6] https://www.geeksforgeeks.org/cpp/range-based-loop-c/

[7] https://en.wikipedia.org/wiki/Particle_decay

[8] https://physics.stackexchange.com/questions/282245/muon-decay-mode

[9] https://en.wikipedia.org/wiki/Muon

[10] https://en.wikipedia.org/wiki/Tau_(particle)

[11] https://tikz.net/sm_decay_matrix/

[12] https://www.geeksforgeeks.org/cpp/stduniform_real_-distribution-class-in-c-with-examples/

[13] https://www.w3schools.com/cpp/cpp_switch.asp

[14] https://www.geeksforgeeks.org/cpp/can-constructor-private-cpp/

[15] https://www.w3schools.com/cpp/cpp_constructors_overloading.asp

[16] https://www.geeksforgeeks.org/system-design/factory-method-pattern-c-design-patterns/

[17] https://pdg.lbl.gov/2020/AtomicNuclearProperties/HTML/tungsten_W.html

[18] https://en.wikipedia.org/wiki/Particle_shower

[19] https://indico.capa.unizar.es/event/23/sessions/48/attachments/287/393/COST-TS-Belgrade-Course5-Tutorial.pdf

[20] https://www.ibm.com/docs/en/dma?topic=diagrams-enumerations

4. Delegation to Generative AI

4.1. Disclosure of Delegation to Generative AI

The author (Kuba Kane) declares the use of generative AI in the research and coding process. The following tasks were delegated to GAI tools under full human supervision:

- Idea generation
- Code generation
- Code optimisation
- Code debugging

The GAI tool used for these tasks was: ChatGPT-5.2, **free version**.

The author declares that the work does not contain any AI-generated code or text. GAI was used as a source of information for explaining code only, and as an inspiration to write original code.

Responsibility for the final manuscript lies entirely with the author.

GAI tools are not listed as authors and do not bear responsibility for the final outcomes.

4.2. GAI prompts

While no AI-generated code was directly used in this project, a list of the prompts used in ChatGPT-5.2 which suggested code relevant to this project can be found below.

- *Shared pointers allow multiple pointers to point to the same object, and only when the last pointer goes out of scope does the memory become freed. Does this mean if you want to change FourMomentum object you need to wait until all pointers which are shared go out of scope?*
- Code generation
- Code optimisation
- Code debugging